# Change Point Detection e Concept Drift em Séries Temporais

CEFET/RJ

Eduardo Ogasawara

eogasawara@cefet-rj.br | https://eic.cefet-rj.br/~eogasawara

# Por que detectar mudanças importa



### Validade de modelos

Modelos de previsão e descoberta de padrões só são válidos enquanto o processo observado permanece semelhante.



### Controle e alerta

Mudanças afetam controle, monitoramento, alerta e adaptação de modelos. Detectar cedo permite ação corretiva.



### Localização temporal

O instante da mudança pode ser tão importante quanto a existência da mudança. Saber quando permite reconstruir a causa.

# Change point vs concept drift

## Change point

É uma observação que marca transição entre dinâmicas distintas da série.

A mudança pode envolver distribuição, tendência, média, variância, volatilidade ou autocorrelação.

Quando a mudança é gradual, o ponto exato pode ser ambíguo.

Foco: localizar o instante ou intervalo em que a dinâmica da série muda.

## Concept drift

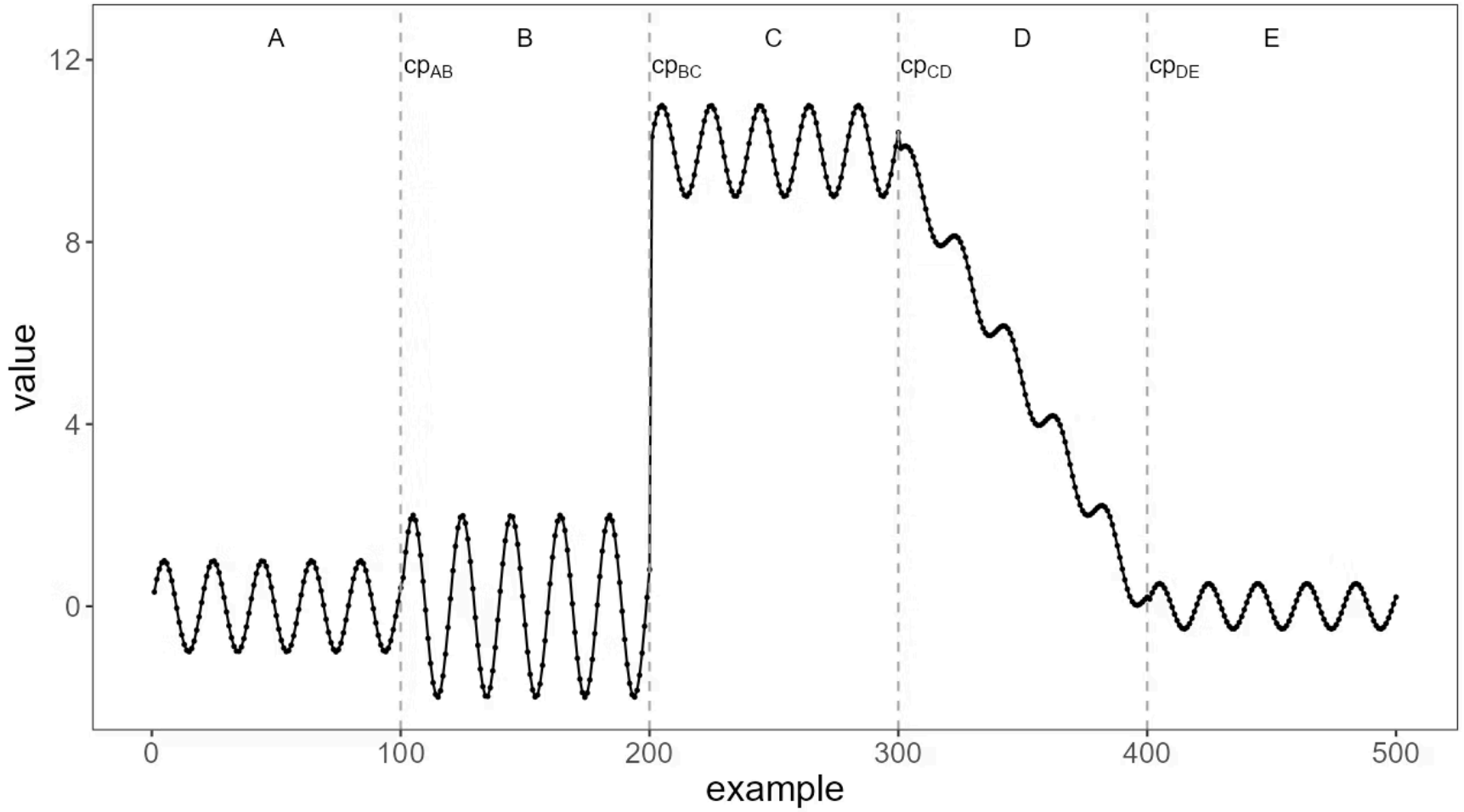
É uma mudança no conceito aprendido ou monitorado ao longo do tempo.

Afeta a validade de modelos treinados em dados históricos.

Foco: monitorar mudanças que afetam a relação entre dados e modelo.

Na prática: change point enfatiza segmentação; drift enfatiza validade e adaptação do modelo.

# Exemplo sintético com quatro change points




A série é dividida em cinco segmentos, com mudanças de volatilidade, tendência e regime.

# Segmentos e dinâmicas distintas

- Cada segmento pode ser interpretado como uma dinâmica temporal própria.
- A detecção busca fronteiras que tornam os segmentos internamente mais homogêneos.
- O problema é separar mudanças reais de flutuações locais.

> "Um bom segmento é aquele em que um único modelo explica bem todos os pontos internos."

### Homogeneidade interna

Dentro de cada segmento, a distribuição ou o modelo deve ser estável. Baixa variância intra-segmento.

### Heterogeneidade entre segmentos

Segmentos adjacentes devem ser estatisticamente distintos. Alta variância inter-segmento.

### Parcimônia

Usar o menor número de segmentos que explique adequadamente os dados. Penalização por excesso de change points.

# Tipos de mudança

## Mudanças bruscas

Transição abrupta entre regimes.

Ponto de mudança mais fácil de localizar.

Comum em falhas, intervenções e quebras estruturais.

## Mudanças graduais

Transição distribuída em um intervalo.

O instante exato é menos evidente.

Pode envolver tendência, deriva ou volatilidade.

# Dimensões do problema de detecção

## Offline vs online

### Offline

A série já foi coletada.

O objetivo é explicar quando as dinâmicas mudaram.

Pode usar informação antes e depois da mudança.

**Métodos:** AMOC, BinSeg, PELT, Chow Test, GFT/CUSUM.

### Online

As observações chegam em fluxo.

A decisão precisa ser rápida e incremental.

Trade-off entre atraso de detecção e falso alarme.

**Métodos:** CUSUM, DDM, ADWIN, Page-Hinkley.

## Número de change points

### Conhecido

O problema é localizar m mudanças.

Mais comum em análise offline controlada.

Permite formular busca por melhores partições.

### Desconhecido

O método precisa decidir quantos segmentos existem.

Superdetecção gera falsos positivos.

Subdetecção gera falsos negativos.

Penalização (BIC, AIC) controla o trade-off.

# Taxonomia de change point detection

## Outline da taxonomia

### Busca por segmentação

- Objetivo: minimizar custo total sobre todos os segmentos.
- Penalização controla número de change points (BIC, AIC, β).
- AMOC: uma única mudança via razão de verossimilhança.
- BinSeg: AMOC recursivo, divisão gulosa.
- PELT: programação dinâmica com poda, solução ótima em O(n).

### Busca por regressão

- Avalia breakpoints via modelos ajustados por segmento.
- Critério: redução de RSS ou ganho de log-verossimilhança.
- Chow Test: estatística F sobre quebra estrutural em regressão linear.
- GFT/CUSUM: resíduos acumulados de modelo ajustado.

### Busca por janelas deslizantes

- Analisa subsequências de tamanho fixo ao longo da série.
- Não exige série completa; aplicável em modo semi-online.
- SCP: ponto seminal fixo no centro da janela, comparação de retas.
- Change Finder: duas fases AR com suavização de scores.

# Busca por segmentação

## Princípio

### Objetivo

Procura partições que tornem cada segmento internamente coerente. Minimiza uma função de custo global sobre todos os segmentos.

### Função de custo

Pode minimizar distância intra-segmento (variância, erro quadrático) ou maximizar verossimilhança por segmento.

### Penalização

Para evitar excesso de segmentos, adiciona penalização por número de change points (BIC, AIC, penalização linear).

## Métodos desta família

### Busca exata

- Programação dinâmica garante solução ótima.
- PELT usa poda para eficiência O(n).
- Custo: depende da função de custo e do número de change points.

### Busca aproximada

- BinSeg aplica divisão gulosa recursiva.
- Não garante ótimo global, mas é eficiente e prático.
- Útil quando n é grande e m é pequeno.

# Busca por regressão

## Princípio

### Objetivo

Avalia breakpoints por meio de modelos ajustados aos segmentos. Um bom breakpoint reduz erro ou melhora a qualidade de ajuste.

### Critério de avaliação

Compara RSS (erro residual) ou log-verossimilhança do modelo único com a soma dos modelos por segmento.

### Varredura

Para cada candidato k, ajusta modelos antes e depois e calcula o ganho. O k ótimo maximiza o ganho.

## Métodos desta família

### Chow Test

- Testa quebra estrutural em modelo de regressão linear.
- Usa estatística F para comparar RSS do modelo único com ESS dos segmentos.
- Requer suposição de normalidade dos resíduos.

### GFT (CUSUM de resíduos)

- Monitora resíduos acumulados de um modelo ajustado.
- Detecta desvio sistemático dos resíduos em relação ao esperado.
- Não exige candidato k fixo: localiza o pico do processo acumulado.

# Busca por janelas deslizantes

## Princípio

### Objetivo

Analisa subsequências de tamanho fixo ao longo da série. Dentro da janela, testa-se se há diferença relevante entre antes e depois de um candidato.

### Vantagem

Não exige que toda a série esteja disponível. Pode ser aplicada em modo online ou semi-online.

### Limitação

O tamanho da janela afeta a sensibilidade. Janelas pequenas detectam mudanças rápidas; janelas grandes detectam mudanças graduais.

## Métodos desta família

### SCP

- Compara regressões dentro de uma janela deslizante.
- Avalia se a divisão interna da janela reduz o erro de ajuste.
- Adequado para mudanças estruturais em modelos lineares.

### Change Finder

- Usa modelo AR para pontuar outliers e depois detecta mudanças nos scores.
- Duas fases: detecção de outliers e detecção de change point.
- Robusto a outliers isolados graças à suavização por média móvel.

# AMOC: intuição

## AMOC: intuição

AMOC procura uma única mudança em toda a série. Cada candidato k divide X em dois segmentos. A pergunta é se os dois segmentos explicam melhor os dados do que um único regime.

$$X_{1:n} \rightarrow X_{1:k} + X_{k+1:n}$$

Estatística monitorada: ganho de verossimilhança ao dividir em k. Parâmetros: limiar γ (gamma).

## Pseudocódigo

ENTRADA: série $X_{1:n}$, limiar γ
PRÉ-CONDIÇÃO: n ≥ 2; espera-se no máximo uma mudança dominante

PASSO 1 — Para cada candidato k de 2 até n−1:
  calcular $L_k$ = logLik($X_{1:k}$) + logLik($X_{k+1:n}$)
PASSO 2 — Calcular $L_0$ = logLik($X_{1:n}$) sem segmentação
PASSO 3 — Calcular estatística:
  $\Lambda = 2 \cdot (\max_k L_k - L_0)$
PASSO 4 — Se Λ > γ:
  k* = argmax_k $L_k$
  retornar k* como change point
  caso contrário: retornar "sem mudança"

SAÍDA: posição k* ou ausência de mudança
PÓS-CONDIÇÃO: AMOC detecta no máximo 1 change point

# AMOC: hipótese nula e alternativa

## H0: sem mudança

A série é explicada por uma única distribuição.

Há um único conjunto de parâmetros θ0.

## H1: mudança em k

Os segmentos antes e depois de k têm distribuições diferentes.

Cada segmento tem seus próprios parâmetros θ1 e θ2.

# AMOC: razão de verossimilhança

## Estatística e decisão

Compara o melhor ajuste com uma distribuição única contra o melhor ajuste dividido em dois segmentos. O k ótimo maximiza o ganho de verossimilhança. A decisão depende de um limiar para rejeitar a hipótese de ausência de mudança.

$$\lambda = 2 \cdot (\max_{k} ML(X, k) - ML(X))$$

Sob $H_0$, λ segue assintoticamente distribuição qui-quadrado com graus de liberdade = diferença de parâmetros entre os modelos.

## Interpretação do ganho

### λ pequeno

Os dois segmentos não explicam melhor os dados do que um único regime. Não há evidência de mudança.

### λ > limiar

O ganho de verossimilhança é estatisticamente significativo. Há evidência de change point em k*.

### Escolha do limiar

Pode ser fixado por distribuição assintótica (qui-quadrado), por permutação ou por critério de informação (AIC/BIC).

# BinSeg: AMOC recursivo

## BinSeg: intuição

BinSeg aplica AMOC recursivamente. A cada iteração, encontra o melhor change point no segmento atual e divide. Repete até atingir critério de parada.

**Função objetivo:** minimizar custo total $C(X_{1:k}) + C(X_{k+1:n})$ em cada divisão.

**Parâmetros:** número máximo de change points *m* ou limiar de ganho.

**Nota:** algoritmo guloso — não garante solução globalmente ótima.

## Pseudocódigo

ENTRADA: série $X_{1:n}$, máx. de change points m (ou limiar λ)
PRÉ-CONDIÇÃO: $n \geq 2$

PASSO 1 — Inicializar: segmentos = $\{X_{1:n}\}$, CPs = {}
PASSO 2 — Enquanto $|CPs| < m$:
  Para cada segmento S em segmentos:
    aplicar AMOC(S) → candidato k*, ganho g*
  Selecionar segmento com maior g*
PASSO 3 — Se $g^* > \lambda$:
  Dividir segmento em $S_1$ e $S_2$ em k*
  Adicionar k* a CPs
  Substituir S por $\{S_1, S_2\}$ em segmentos
  caso contrário: parar
PASSO 4 — Retornar CPs ordenados

SAÍDA: conjunto de change points $\{k_1, k_2, \ldots, k_m\}$
PÓS-CONDIÇÃO: cada segmento é internamente homogêneo

# PELT: segmentação eficiente

## PELT: intuição

PELT busca a segmentação globalmente ótima via programação dinâmica com poda. A poda elimina candidatos que nunca poderão ser ótimos, reduzindo custo computacional.

**Função objetivo:** minimizar $\sum C(\text{segmento}) + \beta \cdot |\text{CPs}|$, onde $\beta$ é a penalização por change point.

**Parâmetros:** função de custo C (ex.: variância, log-verossimilhança negativa), penalização $\beta$.

**Complexidade:** O(n) sob crescimento linear de change points.

## Pseudocódigo

ENTRADA: série $X_{1:n}$, custo $C(\cdot)$, penalização $\beta$
PRÉ-CONDIÇÃO: C satisfaz condição de poda (pruning inequality)

PASSO 1 — Inicializar: $F(0) = -\beta$, $cp(0) = \varnothing$, $R = \{0\}$
PASSO 2 — Para t = 1 até n:
  $F(t) = \min_{s \in R} [\, F(s) + C(X_{s+1:t}) + \beta \,]$
  $s^* = \operatorname{argmin}_{s \in R} [\, F(s) + C(X_{s+1:t}) + \beta \,]$
  $cp(t) = cp(s^*) \cup \{s^*\}$
PASSO 3 — Podar R:
  Para cada $s \in R$:
    Se $F(s) + C(X_{s+1:t}) + \beta > F(t)$:
      remover s de R (nunca será ótimo)
  Adicionar t a R
PASSO 4 — Recuperar change points via $cp(n)$

SAÍDA: conjunto ótimo de change points
PÓS-CONDIÇÃO: solução globalmente ótima sob penalização $\beta$

# Chow Test: mudança estrutural

## Chow Test: intuição

Parte de um modelo de regressão linear: $y_t = \beta_t x_t + \omega_t$

$$y_t = \beta_t x_t + \omega_t$$

Testa se os parâmetros β permanecem estáveis ou mudam ao longo do tempo.

Especialmente útil quando mudança significa quebra estrutural no modelo.

**$H_0$:** $\beta_1 = \beta_2$ (parâmetros iguais antes e depois de k)

**$H_1$:** $\beta_1 \neq \beta_2$ (quebra estrutural em k)

## Pseudocódigo

ENTRADA: série $(y_t, x_t)$, candidato k, nível α
PRÉ-CONDIÇÃO: resíduos aproximadamente normais e homocedásticos

PASSO 1 — Ajustar modelo completo sobre $X_{1:n}$:
$\hat{y}_t = \hat{\beta} \cdot x_t$
$RSS = \Sigma(y_t - \hat{y}_t)^2$
PASSO 2 — Ajustar modelo no segmento 1 ($t \leq k$):
$\hat{y}_t^{(1)} = \hat{\beta}_1 \cdot x_t$
$RSS_1 = \Sigma_{t \leq k} (y_t - \hat{y}_t^{(1)})^2$
PASSO 3 — Ajustar modelo no segmento 2 ($t > k$):
$\hat{y}_t^{(2)} = \hat{\beta}_2 \cdot x_t$
$RSS_2 = \Sigma_{t > k} (y_t - \hat{y}_t^{(2)})^2$
$ESS = RSS_1 + RSS_2$
PASSO 4 — Calcular estatística F:
$F = [(RSS - ESS)/r] / [ESS/(n - 2r)]$
onde r = número de parâmetros
PASSO 5 — Se F > F_crítico(α): rejeitar $H_0$ → mudança em k

SAÍDA: decisão de quebra estrutural em k
PÓS-CONDIÇÃO: repetir para múltiplos k (varredura)

# Chow Test: estatística F

## Estatística F e interpretação

Ajusta um modelo para toda a série e modelos separados antes/depois do candidato. Compara RSS e ESS para avaliar se a divisão reduz erro de forma significativa. Um F alto indica evidência de mudança estrutural.

$$F = \frac{(RSS - ESS)}{ESS/(n - 2r)}$$

Onde **RSS** = soma dos quadrados dos resíduos do modelo único; **ESS** = $RSS_1 + RSS_2$ dos modelos separados; **r** = número de parâmetros; **n** = tamanho da série.

Sob $H_0$, F ~ F(r, n−2r). Rejeitar $H_0$ se F > F_crítico(α).

## Interpretação dos componentes

### RSS (modelo único)

Erro residual ao ajustar um único modelo para toda a série. Representa o custo de ignorar a possível quebra.

### ESS (modelos separados)

Soma dos erros residuais dos dois segmentos. Representa o custo de permitir a quebra em k.

## Ganho de ajuste

RSS − ESS mede o quanto a divisão reduz o erro. Normalizado por ESS/(n−2r) para controlar graus de liberdade.

# GFT/CUSUM: resíduos acumulados

## GFT/CUSUM: intuição

Ajusta um modelo de regressão e monitora os resíduos acumulados. Sob estabilidade, o processo acumulado fica dentro de limites esperados. Sob mudança, apresenta pico próximo ao change point.

**Estatística monitorada:**

$$W_n^0(u) = \frac{1}{\hat{\sigma}\sqrt{n}} \cdot \sum_{i=1}^{\lfloor nu \rfloor} \hat{\omega}_i$$

**Parâmetros:** modelo de regressão base, limiar de significância.

## Pseudocódigo

ENTRADA: série $X_{1:n}$, modelo de regressão M, limiar h
PRÉ-CONDIÇÃO: M ajustado em período estável de referência

PASSO 1 — Ajustar M sobre $X_{1:n}$ → obter resíduos $\hat{\omega}_1, \ldots, \hat{\omega}_n$
PASSO 2 — Estimar desvio padrão $\hat{\sigma}$ dos resíduos
PASSO 3 — Para cada $u \in (0,1]$:
  Calcular processo acumulado:
  $W(u) = (1 / \hat{\sigma}\sqrt{n}) \cdot \Sigma_{i=1}^{\lfloor nu \rfloor} \hat{\omega}_i$
PASSO 4 — Calcular estatística de teste:
  $\sup_u |W(u)|$ ou variante funcional
PASSO 5 — Se estatística > h:
  Localizar $u^* = \text{argmax}_u |W(u)|$
  Reportar change point em $\lfloor nu^* \rfloor$
  caso contrário: série estável

SAÍDA: posição do change point ou ausência de mudança
PÓS-CONDIÇÃO: sensível a quebras estruturais no modelo

# SCP: estratégia com janela deslizante

## Intuição

SCP desliza uma janela de tamanho fixo sobre a série. O ponto central de cada janela é o candidato fixo — o ponto seminal. Compara o erro de uma regressão linear sobre a janela inteira com o erro de duas regressões separadas: antes e depois do centro. A diferença de erros é o score de mudança. Change points são os scores identificados como outliers pelo filtro boxplot.

**Estatística monitorada:**

$$\text{score}_t = \overline{|r_{\text{full}}|} - \overline{|r_{\text{split}}|}$$

**Parâmetros:**

- sw_size: tamanho da janela (padrão 30)
- offset = ⌊sw_size / 2⌋: posição do ponto seminal
- Limiar automático: Q3 + 1,5 · IQR

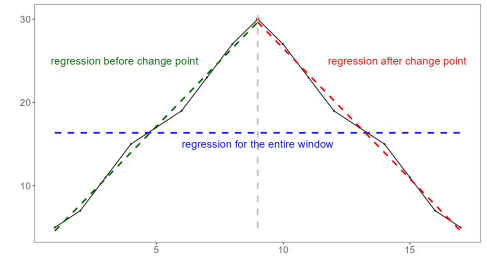
## Pseudocódigo

ENTRADA: série $X_{1:n}$, sw_size
PRÉ-CONDIÇÃO: sw_size ≥ 4; offset = ⌊sw_size / 2⌋

PASSO 1 — Para cada janela W de tamanho sw_size:
  Ajustar reta sobre W inteira
  err_full = média dos |resíduos|
PASSO 2 — Dividir W no ponto seminal (posição offset):
  Ajustar reta em W[1 : offset−1] → err_esq
  Ajustar reta em W[offset+1 : sw_size] → err_dir
  err_split = média de (err_esq + err_dir)
PASSO 3 — Calcular score da janela:
  $\text{score}_t$ = err_full − err_split
PASSO 4 — Após varrer todas as janelas:
  limiar = Q3(scores) + 1,5 · IQR(scores)
  change_point = {t : $\text{score}_t$ > limiar}
PASSO 5 — Alinhar índices:
  padding com zeros nas bordas (offset à esquerda,
  sw_size − offset − 1 à direita)

SAÍDA: posições t onde $\text{score}_t$ > limiar
PÓS-CONDIÇÃO: cada CP é o centro da janela
  com maior contraste de ajuste local

## Ilustração




Compara o ajuste de uma regressão única com regressões separadas antes e depois do candidato.

# Change Finder: duas fases

## Change Finder: intuição

Detecta change points em duas fases. Fase 1: modela a série com AR e pontua outliers via log-verossimilhança. Fase 2: suaviza os scores e aplica um segundo modelo AR para detectar mudanças no padrão de scores.

**Estatística monitorada:** Fase 1: score de outlier $s_t = -\log p(x_t \mid x_{t-1}, \ldots, x_{t-p})$, suavizado em $\bar{s}_t$. Fase 2: score $a_t = -\log p(\bar{s}_t \mid \bar{s}_{t-1}, \ldots, \bar{s}_{t-p})$, suavizado em $\bar{a}_t$.

**Parâmetros:** ordem p do AR, janela de suavização w, limiar de alarme.

## Pseudocódigo

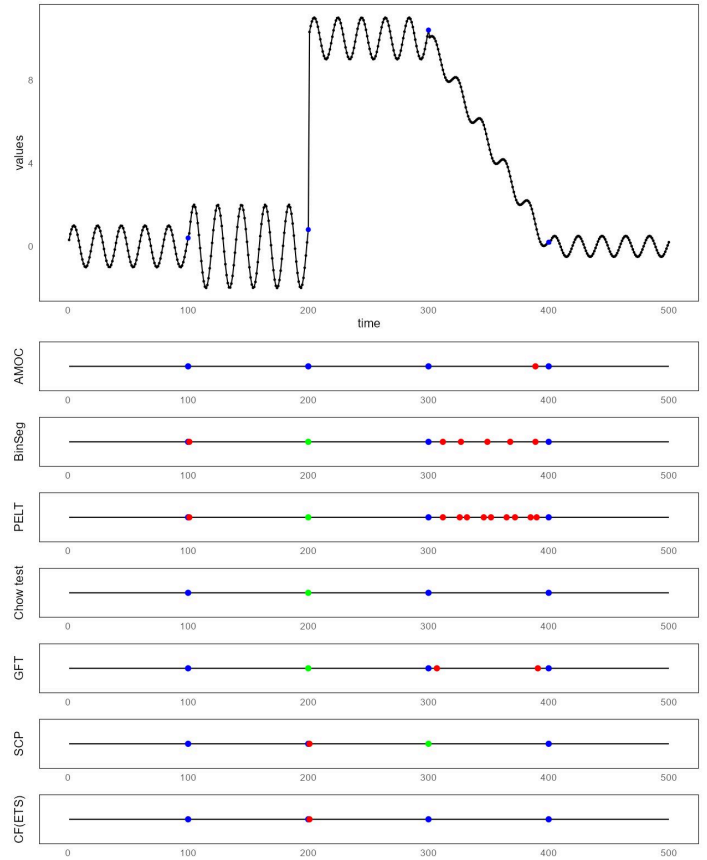
ENTRADA: série $X_{1:n}$, ordem AR p, janela w, limiar h
PRÉ-CONDIÇÃO: n > p + w

FASE 1 — Detecção de outliers:
PASSO 1 — Ajustar AR(p) sobre $X_{1:n}$
PASSO 2 — Para cada t > p:
  $s_t = -\log p(x_t \mid x_{t-1}, \ldots, x_{t-p})$
PASSO 3 — Suavizar: $\bar{s}_t = (1/w) \cdot \Sigma_{i=0}^{w-1} s_{t-i}$

FASE 2 — Detecção de change point:
PASSO 4 — Ajustar AR(p) sobre $\bar{s}_{1:n}$
PASSO 5 — Para cada t:
  $a_t = -\log p(\bar{s}_t \mid \bar{s}_{t-1}, \ldots, \bar{s}_{t-p})$
PASSO 6 — Suavizar: $\bar{a}_t = (1/w) \cdot \Sigma_{i=0}^{w-1} a_{t-i}$
PASSO 7 — Se $\bar{a}_t > h$: sinalizar change point em t

SAÍDA: instantes t com $\bar{a}_t > h$
PÓS-CONDIÇÃO: robusto a outliers isolados

# Comparação dos detectores de change point




Verde indica acerto, azul indica evento perdido e vermelho indica detecção incorreta.

# O que é concept drift

### Definição

É uma mudança no conceito aprendido ou monitorado ao longo do tempo. Afeta a validade de modelos treinados em dados históricos.

### Origem

Pode surgir de mudanças no comportamento do usuário, no ambiente, nas políticas ou nos próprios dados coletados.

### Consequência

O modelo passa a cometer mais erros sem que seus parâmetros internos mudem. A degradação pode ser gradual ou abrupta.

### Diferença de change point

Change point foca na série temporal em si. Concept drift foca na relação entre dados e conceito aprendido — especialmente em modelos preditivos.

### Quando monitorar

Sempre que um modelo é implantado em produção e os dados continuam chegando. Qualquer sistema de ML em produção está sujeito a concept drift.

# Conceito como distribuição e drift

## Conceito como distribuição

Considere dados D organizados em batches.

O conceito em um batch é representado pela distribuição associada aos dados observados.

Mudança de conceito significa mudança estatisticamente relevante nessa distribuição.

$$\text{concept}(SD_i) = p(\chi_i)$$

## Drift entre batches

Entre batches consecutivos, o drift aparece quando as distribuições deixam de ser equivalentes.

A diferença pode ocorrer nos dados de entrada, na relação X → Y ou no erro do modelo.

$$p(\chi_i) \neq p(\chi_{i+1})$$

### Drift de entrada (virtual)

Muda p(X). A relação X→Y pode permanecer válida.

### Drift real

Muda p(Y|X). O modelo perde validade preditiva e exige adaptação.

# Taxonomia de concept drift

### Tipos de drift

- **Virtual:** muda p(X), relação X→Y pode ser válida.
- **Real:** muda p(Y|X), modelo perde validade.
- Abrupto, gradual, incremental, recorrente.

### Tipos de detector

- **Baseados em erro:** monitoram desempenho do modelo (DDM, HDDM, ECDD).
- **Baseados em distribuição:** comparam janelas de dados (ADWIN, KSWIN, KLD, MCDD).

### Métodos

- **Incrementais:** CUSUM, Page-Hinkley, DDM, ADWIN.
- **Por batch:** MCDD, KLD, KSWIN.
- **Ensemble:** combinam múltiplos detectores ou modelos.

### Tratamento

- Retreinamento completo após alarme.
- Atualização incremental contínua.
- Ensemble adaptativo com ponderação por desempenho.

A taxonomia organiza tipos de drift, tipos de detector, métodos e formas de tratamento.

# Drift real vs virtual

## Virtual drift

A distribuição dos dados de entrada muda.

A relação com a variável-alvo pode permanecer válida.

Pode afetar representatividade e amostragem.

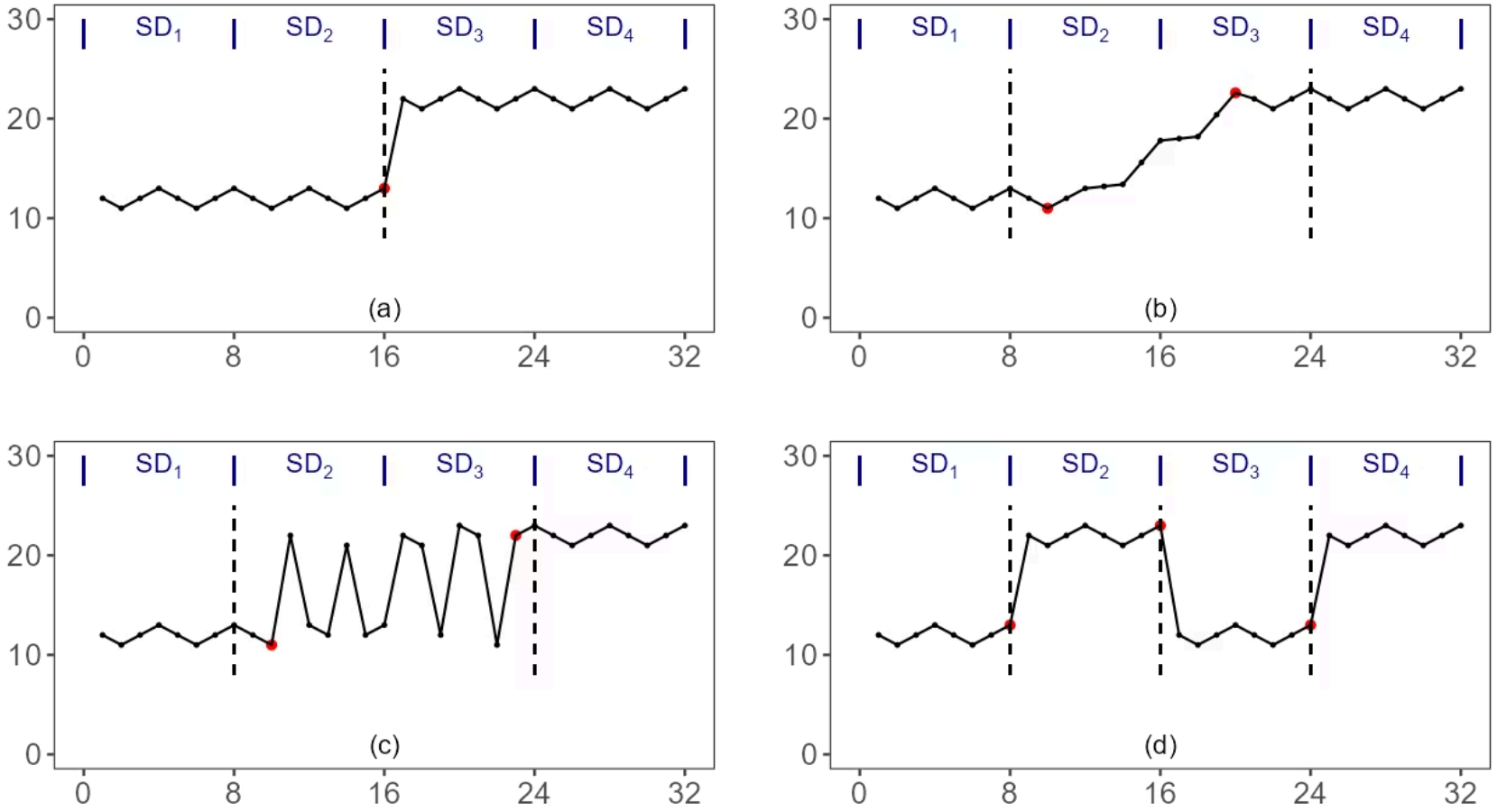
## Real drift

A relação entre entrada e saída muda.

O modelo perde validade preditiva.

Normalmente exige adaptação ou retreinamento.

# Tipos de drift




Drifts podem ser abruptos, incrementais, graduais ou recorrentes.

# Detectar mudança vs adaptar modelo

## Detectar drift

- Responde: o conceito mudou?
- Monitora estatísticas do fluxo ou do erro do modelo.
- Produz um sinal de alarme com localização temporal.
- Sensibilidade vs especificidade: trade-off central.

## Adaptar o modelo

- Responde: o que fazer depois da mudança?
- Pode envolver retreinamento completo, atualização incremental ou seleção de modelo.
- Depende de boa detecção para não adaptar desnecessariamente.

### Detector sensível demais

Causa falso alarme. O modelo é retreinado sem necessidade, absorvendo ruído como se fosse sinal.

### Detector conservador demais

Reage tarde. O modelo opera com conceito desatualizado, acumulando erro antes do alarme.

### Equilíbrio ideal

Calibrar limiar de alarme com base em latência aceitável, custo de retreinamento e risco de falso alarme.

# Tipos de detector de drift

### Baseados em erro

Monitoram queda de desempenho do modelo.

DDM, HDDM, CUSUM e ECDD seguem essa linha.

Dependem de rótulo ou feedback.

### Baseados em distribuição

Comparam batches, janelas ou amostras.

MCDD, KLD, KSWIN, Page-Hinkley e ADWIN são exemplos.

Podem atuar antes do impacto no erro.

# Critérios de escolha

### Disponibilidade de rótulos

Detectores baseados em erro exigem rótulos verdadeiros ($y_t$) em tempo real. Se rótulos chegam com atraso, prefira detectores baseados em dados.

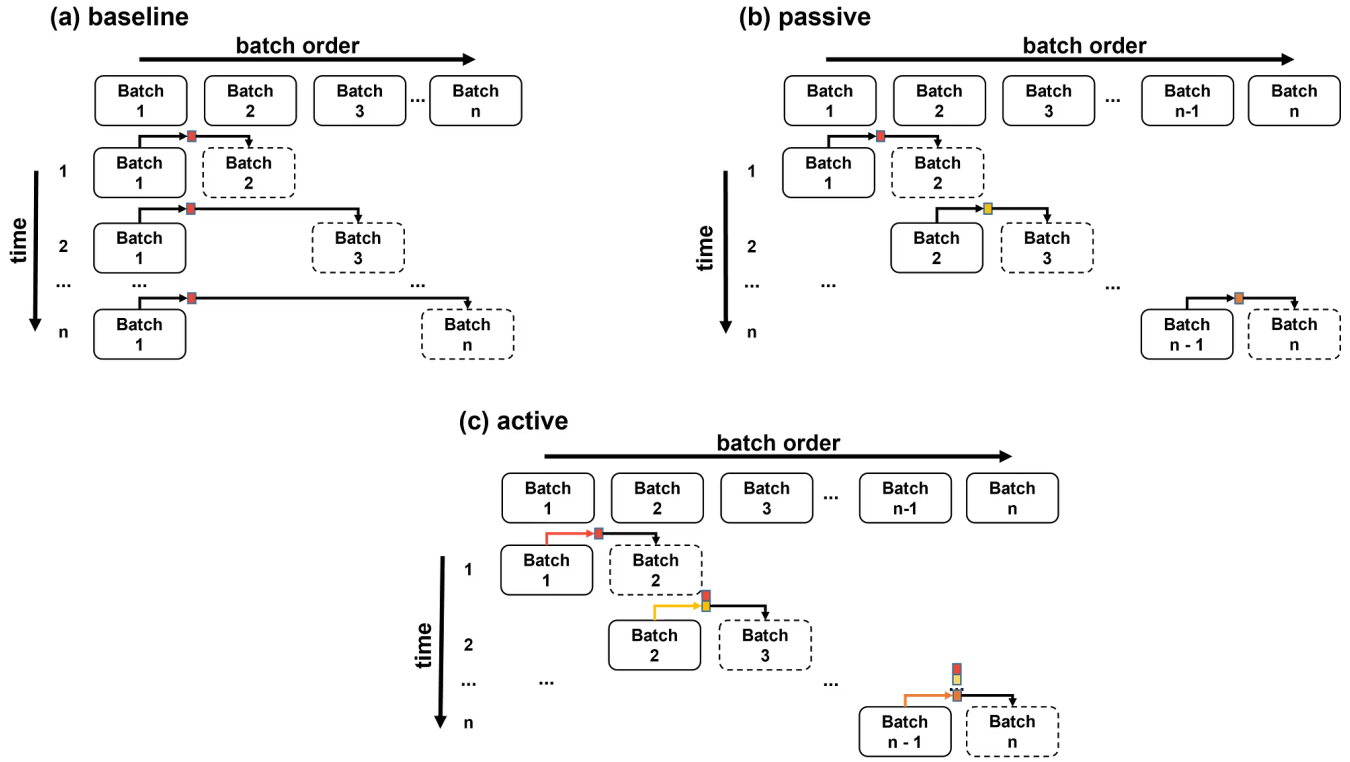
### Tipo de drift esperado

Drift de média → CUSUM, Page-Hinkley. Drift de distribuição → KSWIN, KLD. Drift de erro de classificador → DDM, HDDM, ECDD.

### Custo computacional

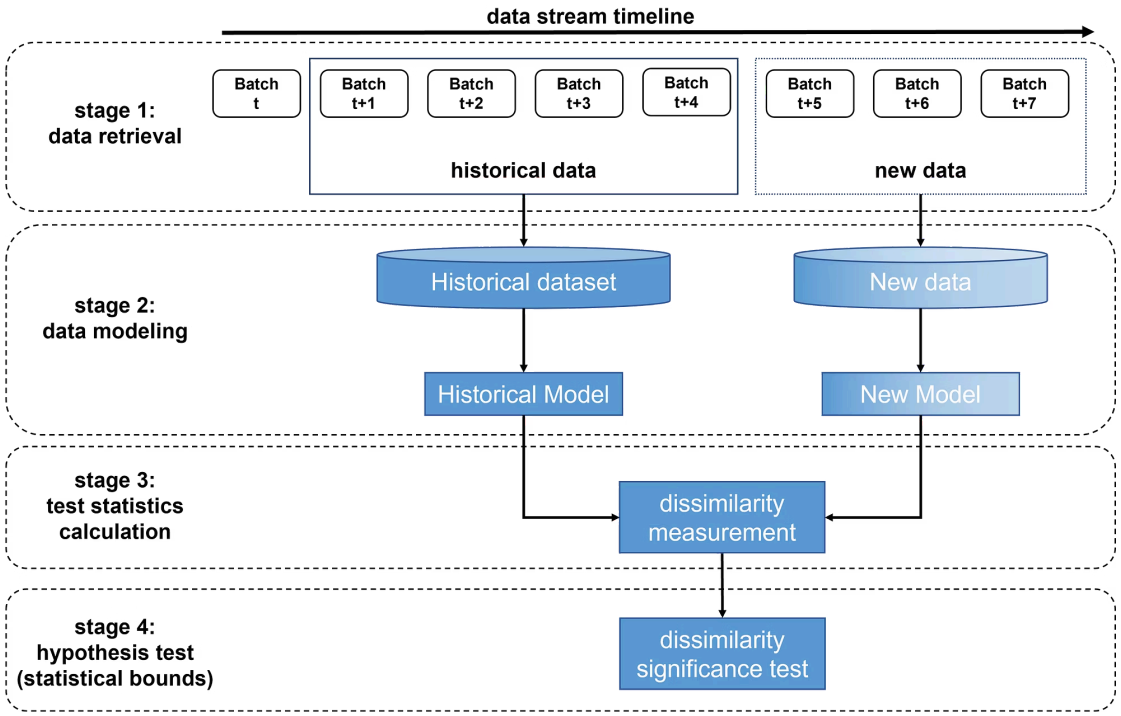
Detectores incrementais (CUSUM, DDM, ADWIN) são adequados para fluxos de alta velocidade. Detectores por batch (MCDD, KLD) são mais robustos mas exigem acumulação de dados.

# Tratamento de drift




Baseline não reage, passive atualiza continuamente e active atualiza após detecção.

# Framework geral de drift detection




O framework combina dados em fluxo, monitoramento, detecção e eventual adaptação do modelo.

# Quando atualizar o modelo

## Atualização contínua

- Acompanha mudanças graduais sem necessidade de detector explícito.
- Risco: absorver ruído e flutuações como se fossem mudanças reais.
- Adequado quando o conceito muda lentamente e o custo de retreinamento é baixo.

## Atualização após alarme

- Reduz custo computacional: retreina apenas quando necessário.
- Depende de boa detecção para não reagir tarde ou em falso alarme.
- Adequado quando o conceito muda abruptamente e o retreinamento é caro.

## Ensemble adaptativo

- Mantém múltiplos modelos treinados em diferentes janelas temporais.
- Pondera modelos pelo desempenho recente.
- Robusto a diferentes velocidades de drift.

### Latência aceitável

Quanto tempo o modelo pode operar com conceito desatualizado antes de causar dano?

### Custo de retreinamento

Retreinar é caro? Prefira atualização incremental ou ensemble.

### Risco de falso alarme

Um alarme falso leva a retreinamento desnecessário. Calibrar o detector para o contexto.

# DDM e HDDM

## DDM e HDDM: intuição

**DDM:** monitora a taxa de erro $p_t$ e seu desvio padrão $s_t = \sqrt{(p_t(1-p_t)/t)}$. Alarme quando $p_t + s_t > p_min + k \cdot s_min$.

**HDDM:** usa desigualdades de Hoeffding para controlar detecção com limites probabilísticos. Mais robusto a flutuações transitórias.

**Parâmetros DDM:** *k_warning* (nível de aviso), *k_alarm* (nível de alarme).

**Parâmetros HDDM:** *δ_warning*, *δ_alarm* (probabilidades de erro tipo I).

ENTRADA: fluxo de predições ($\hat{y}_t$, $y_t$), parâmetros k_w, k_a
PRÉ-CONDIÇÃO: classificador treinado em conceito inicial

PASSO 1 — Inicializar: p_min = ∞, s_min = ∞, t = 0
PASSO 2 — Para cada nova observação ($\hat{y}_t$, $y_t$):
  $e_t$ = 1 se $\hat{y}_t \neq y_t$, senão 0
  Atualizar $p_t$ = média acumulada de $e_t$
  Calcular $s_t = \sqrt{(p_t(1 - p_t) / t)}$
PASSO 3 — Atualizar mínimos:
  Se $p_t + s_t <$ p_min + s_min:
    p_min = $p_t$, s_min = $s_t$
PASSO 4 — Verificar limiares:
  Se $p_t + s_t >$ p_min + k_a · s_min:
    ALARME: sinalizar drift, reiniciar modelo
  Se $p_t + s_t >$ p_min + k_w · s_min:
    AVISO: acumular dados recentes

SAÍDA: instantes de alarme e aviso
PÓS-CONDIÇÃO: HDDM substitui k por limites de Hoeffding

# Hoeffding bound: intuição

## Intuição e fórmula

- Fornece um limite para diferenças esperadas entre estimativas amostrais.
- Se a diferença observada ultrapassa o limite, há evidência estatística de mudança.
- É útil em fluxo porque permite decisão incremental.

$$\Pr(|\bar{x} - E[x]| \geq \varepsilon) \leq 2\exp(-2n\varepsilon^2)$$

**Parâmetros:** *n* (tamanho da amostra), *ε* (margem de erro tolerada).

## Aplicação em detecção de drift

### Coletar amostra

Acumular *n* observações do fluxo atual.

### Calcular ε_n

Derivar o limiar de Hoeffding para *n* e *δ* desejado: *ε = √(ln(1/δ) / 2n)*.

### Comparar médias

Se |x̄_recente − x̄_referência| > ε_n, sinalizar drift com confiança 1−δ.

# CUSUM: evidência acumulada

## CUSUM: intuição

Acumula desvios sucessivos em relação ao comportamento esperado. Quando a soma acumulada ultrapassa um limiar, sinaliza mudança. Simples, incremental e sensível a mudanças persistentes.

**Estatística monitorada:**

$$z_t = \max(0,\ z_{t-1} + \omega_t - \delta)$$

**Parâmetros:**

- **δ** — magnitude mínima de mudança a detectar
- **h** — limiar de alarme

Reinicia $z_t = 0$ após alarme.

ENTRADA: fluxo $x_1, x_2, \ldots$, média esperada $\mu_0$, $\delta$, limiar $h$
PRÉ-CONDIÇÃO: série estacionária sob $H_0$

PASSO 1 — Inicializar: $z_0 = 0$, $t = 1$
PASSO 2 — Para cada nova observação $x_t$:
  Calcular desvio: $\omega_t = x_t - \mu_0$
  Atualizar estatística:
    $z_t = \max(0, z_{t-1} + \omega_t - \delta)$
PASSO 3 — Se $z_t > h$:
  Sinalizar mudança no instante t
  Reiniciar: $z_t = 0$ (opcional)
  caso contrário: continuar
PASSO 4 — Incrementar t → voltar ao Passo 2

SAÍDA: instantes t em que $z_t > h$
PÓS-CONDIÇÃO: detecta mudanças persistentes de média

# ECDD e MCDD

## ECDD

Usa EWMA para suavizar a taxa de erro:

$$Z_t = \lambda \cdot e_t + (1 - \lambda) \cdot Z_{t-1}$$

Detecta quando o seguidor se afasta do limite de controle.

Dá inércia ao monitoramento, reduzindo falsos alarmes.

**Parâmetros:**

- **λ** — fator de suavização EWMA
- **L** — largura do limite de controle

**Alarme quando:**

$$Z_t > \mu_Z + L \cdot \sigma_Z$$

## MCDD

Compara batches recentes e anteriores via estatísticas multivariadas.

Avalia média e variância de variáveis preditoras.

Pode usar testes paramétricos (**Hotelling $T^2$**) ou não paramétricos (**MMD**).

**Parâmetros:**

- **tamanho do batch** — janela de comparação
- **α** — nível de significância

Útil quando o drift afeta múltiplas variáveis simultaneamente.

# Page-Hinkley e KSWIN

## Page-Hinkley

- Variação do CUSUM que acumula diferenças em relação à média corrente.
- Estatística: PHt = $\Sigma_{i=1}^{t}$ ($x_i - \bar{x}_t - \delta$), alarme quando $PH_t - \min(PH) > h$.
- Funciona bem para mudanças abruptas de média.
- Parâmetros: δ (magnitude mínima), h (limiar).

—— PAGE-HINKLEY ——
ENTRADA: fluxo $x_t$, δ, limiar h
PASSO 1 — Inicializar: $PH_0 = 0$, $m_0 = 0$, $\bar{x}_0 = 0$
PASSO 2 — Para cada $x_t$:
  Atualizar média: $\bar{x}_t = \bar{x}_{t-1} + (x_t - \bar{x}_{t-1})/t$
  $PH_t = PH_{t-1} + (x_t - \bar{x}_t - \delta)$
  $m_t = \min(m_{t-1}, PH_t)$
  Se $PH_t - m_t > h$: ALARME em t

## KSWIN

- Usa teste Kolmogorov-Smirnov sobre duas janelas deslizantes.
- Compara distribuições empíricas de W_r (referência) e W_n (recente).
- Não exige suposição paramétrica forte.
- Parâmetros: tamanho da janela n, tamanho da subjanela r, α (nível de significância).

—— KSWIN ——
ENTRADA: fluxo $x_t$, janela n, subjanela r, nível α
PASSO 1 — Manter janela deslizante W de tamanho n
PASSO 2 — Para cada nova $x_t$:
  Adicionar $x_t$ a W; remover elemento mais antigo
  W_r = últimos r elementos de W
  W_ref = W \ W_r (elementos de referência)
PASSO 3 — Calcular estatística KS:
  D = sup_x |F_ref(x) − F_r(x)|
  p-valor via distribuição KS
PASSO 4 — Se p-valor < α: ALARME em t

SAÍDA: instantes de alarme para cada método

# KLD: divergência entre distribuições

## Definição e fórmula

- Compara uma distribuição antiga P com uma distribuição recente Q.
- Mede o excesso de surpresa ao usar a distribuição esperada para explicar os dados atuais.
- Drift é indicado quando a divergência ultrapassa um limiar.

$$D_{KL}(P\|Q) = \sum P(x_i) \cdot \ln\left(\frac{P(x_i)}{Q(x_i)}\right)$$

KLD não é simétrica: D_KL(P‖Q) ≠ D_KL(Q‖P). Para detecção bidirecional, usar Jensen-Shannon divergence.

## Aplicação prática

### Estimação das distribuições

- Estimar P a partir de uma janela de referência (dados históricos).
- Estimar Q a partir de uma janela recente do fluxo.
- Usar histogramas ou estimação por kernel (KDE).

### Decisão de drift

- Calcular D_KL(P‖Q) para cada novo batch.
- Comparar com limiar h calibrado por bootstrap ou validação cruzada.
- Se D_KL > h: sinalizar drift e atualizar P ← Q.

# ADWIN: janela adaptativa

## ADWIN: intuição

**Explicação:** Mantém uma janela $W$ de tamanho variável sobre o fluxo. Divide $W$ em duas subjanelas $W_0$ (antiga) e $W_1$ (recente) e testa diferença estatística via Hoeffding bound. Quando detecta mudança, descarta $W_0$ e mantém $W_1$.

**Estatística monitorada:** $|\mu(W_0) - \mu(W_1)| > \varepsilon_cut$, onde $\varepsilon_cut$ depende do tamanho das janelas e de $\delta$.

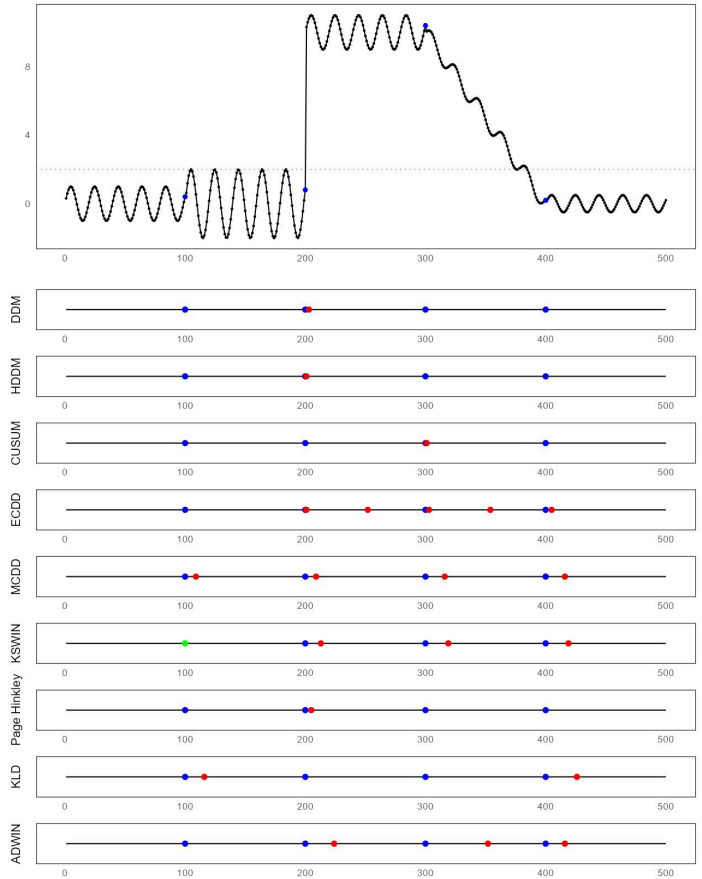
**Parâmetros:** $\delta$ (confiança estatística), tamanho mínimo de subjanela.

ENTRADA: fluxo $x_1, x_2, \ldots$, confiança $\delta$
PRÉ-CONDIÇÃO: $\delta \in (0,1)$

PASSO 1 — Inicializar: $W = []$, $t = 0$
PASSO 2 — Para cada nova observação $x_t$:
  Adicionar $x_t$ ao final de $W$
PASSO 3 — Para cada divisão $W = W_0 \cup W_1$:
  Calcular $\hat{\mu}_0 = \text{média}(W_0)$, $\hat{\mu}_1 = \text{média}(W_1)$
  Calcular limiar de Hoeffding:
    $\varepsilon_cut = \sqrt{( (1/2)|W_0| + 1/2)|W_1|) \cdot \ln(4|W|/\delta)}$
  Se $|\hat{\mu}_0 - \hat{\mu}_1| \geq \varepsilon_cut$:
    Sinalizar drift
    Descartar $W_0$: $W \leftarrow W_1$
    Reiniciar busca
PASSO 4 — Incrementar $t \rightarrow$ voltar ao Passo 2

SAÍDA: instantes de drift e janela W atualizada
PÓS-CONDIÇÃO: W sempre representa o conceito mais recente

# Comparação dos detectores de drift




Verde indica acerto, azul indica evento perdido e vermelho indica detecção incorreta.

# Séries multivariadas e métodos espectrais

## Desafios multivariados

### Dimensionalidade

Mais dimensões aumentam a dificuldade de localizar e explicar mudanças. A maldição da dimensionalidade afeta testes estatísticos.

### Mudanças parciais

Mudanças podem ocorrer em apenas um subconjunto de variáveis. Detectores univariados aplicados separadamente perdem correlações.

### Dependência espacial

Em séries espaço-temporais, mudanças podem se propagar geograficamente. A localização tem dimensão espacial além da temporal.

## Abordagens

### Redução de dimensionalidade

PCA, autoencoders e projeções aleatórias comprimem a série multivariada. Detectores univariados são aplicados no espaço reduzido.

### Testes multivariados

Hotelling $T^2$, MMD (Maximum Mean Discrepancy) e testes de energia comparam distribuições multivariadas diretamente. Mais poderosos, mas computacionalmente mais custosos.

### Métodos espectrais

PCA e autoencoders capturam representações compactas. Drifts aparecem como mudança na reconstrução ou no espaço latente. Promissores para séries não estacionárias e em fluxo.

# Desafios práticos

### Atraso

Detectar cedo o suficiente para agir. Há um trade-off entre velocidade de detecção e taxa de falso alarme. Detectores mais sensíveis reagem mais rápido, mas também a ruído.

### Falso alarme

Evitar adaptação desnecessária. Um alarme falso leva a retreinamento sem necessidade, podendo degradar o modelo. Calibrar o limiar com dados de validação.

### Custo

Manter monitoramento e retreinamento viáveis. Em sistemas de alta frequência, o custo computacional do detector deve ser compatível com a latência exigida.

### Interpretação

Explicar que tipo de mudança ocorreu. Saber se a mudança é na entrada, na relação X→Y ou no erro do modelo orienta a estratégia de adaptação.

### Dados rotulados escassos

Em produção, rótulos verdadeiros chegam com atraso ou não chegam. Detectores baseados em distribuição são preferíveis nesses cenários.

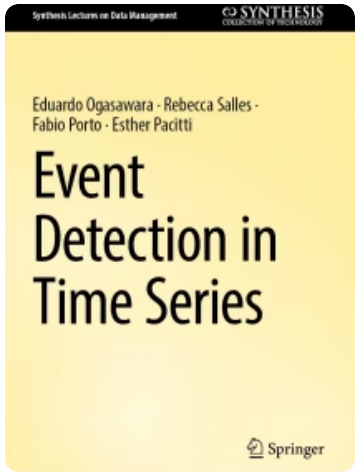
### Mudanças recorrentes

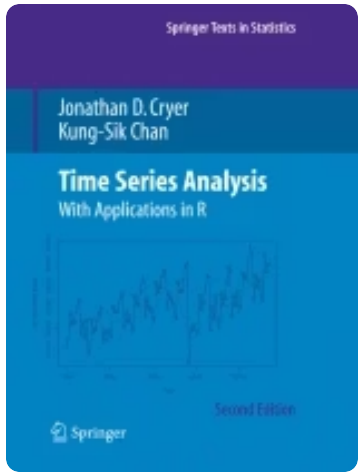
Conceitos podem retornar (concept recurrence). Armazenar modelos antigos e reativá-los pode ser mais eficiente que retreinar do zero.

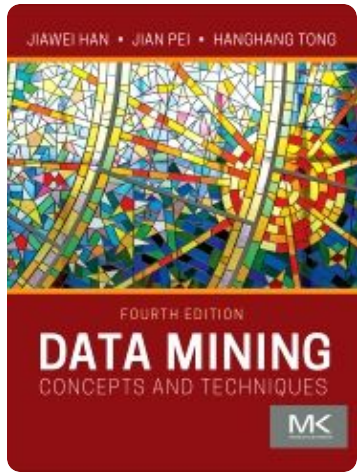
### Monitoramento contínuo

Implementar pipelines de monitoramento com alertas automáticos, dashboards de desempenho e logs de detecção para auditoria.

# Referências Bibliográficas





### Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

### Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

### Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4ª ed., 2022.